Diagnose Before Solving: AI Simulation

Technical Design (v1.3)

Project owner: David Bacon
Document status: Revised for publication
Date: August 31, 2026 (v1.0); revised November 11, 2026 (v1.1), and November 12, 2026 (v1.2 and v1.3)
Deliverable: 6 of 11
Depends on: Scope Document v1.2, Design and Prompt Architecture v1.1, Evaluation Design v1.5

> **Revision note (v1.1, November 11, 2026).** No Storyline module embeds the simulation any more: the pre-simulation training is a separate static site that links to it (D-173, D-177). The embed capability was kept as a verified feature (D-188), so this document now treats Storyline as one possible host page among others. Changed: the glossary entries and system-shape diagram (section 2, 3), section 7's opening, section 10 (retitled), section 11 (retitled), sections 15, 15.1, 15.2, 18, 19 and 20. **No design, header, payload, or code decision changed.** The application is as built. Section 5, 6, 8, 9, 12, 13, 14, 16, 17 and the appendix are unchanged.

> **Revision note (v1.2, November 12, 2026).** Brings the document into line with what is deployed. Section 5.3 is relabelled as the design-time model it always was, and 5.3a gains the later measurements and the price correction (D-131, D-139). Section 7 describes how the session ID now survives a reload, and section 12 no longer says scores and feedback are not stored (D-141). Section 8 adds the daily sub-ceiling and uses the real spend key names (D-128, D-138). Section 10 describes the framing header as implemented (D-200 to D-202). Section 15 lists every configuration value now in use, and 15.1 the deployed `ALLOWED_FRAME_ANCESTORS`. Section 16 matches the repository. Section 20 is updated. No design decision changes.

> **Revision note (v1.3, November 12, 2026).** Section 11 said the completion message specifies a target origin rather than `*`, but the code still posted to `*`. The code now matches the document (D-219): the target is `https://dbacon6093.github.io` by default, overridable by `NEXT_PUBLIC_COMPLETION_TARGET_ORIGIN`. Changed: section 11 (implementation note), section 15 (the new variable), section 16 (the two small prompt note files that were missing from the layout). No design decision changes.

---

1. How to read this document

This document decides how the application is built and hosted. It is written to be understood by the person maintaining it rather than by a backend engineer, which means concepts are explained before they are used and jargon is defined on first appearance.

That is a deliberate choice with a practical purpose. Every technical decision here eventually has to be defended in an interview or fixed at eleven o'clock at night eighteen months from now. A decision the owner cannot explain is a decision that will be reversed by accident.

Three constraints shape everything below, and they come from the Scope Document rather than from technical preference:

1. **The API key must never reach the browser.** This is the one non-negotiable.
2. **Spending must be capped, and hitting the cap must not break the page.** This is the failure that killed the devlin.ai version.
3. **The result must be maintainable by one person with limited backend experience.** Sophistication that cannot be maintained is a future outage.

Section 18 says plainly which parts the owner should expect to change without help and which parts should not be hand-edited.

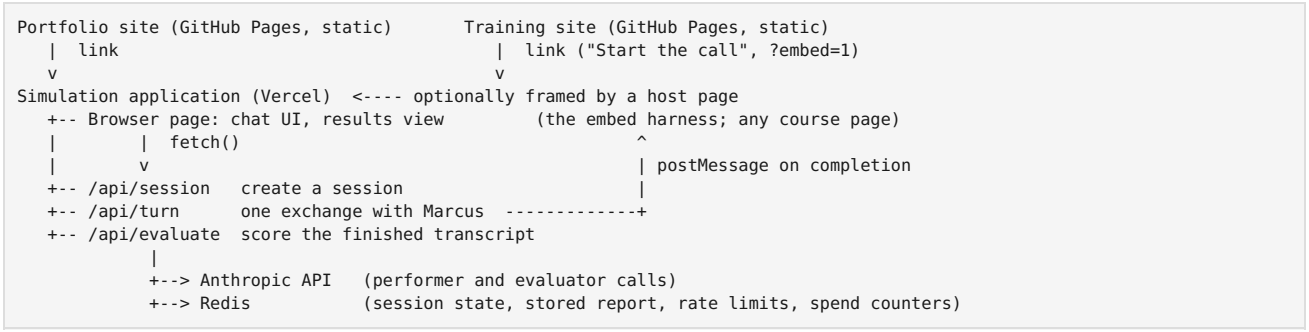
---

2. Concepts used in this document

Term	What it means here
Serverless function	A small piece of code that runs on a server only when a request arrives. There is no machine to keep running or patch. It starts, handles one request, and stops
API route	One such function, reachable at a URL like `/api/turn`
Environment variable	A setting stored in the hosting dashboard rather than in the code, so secrets never appear in the repository. The API key lives here
iframe	A web page embedded inside another page. A Storyline web object is an iframe, and so is the embed harness's frame
`frame-ancestors`	A response header telling the browser which sites are allowed to embed this page in an iframe. If it is present and does not list a host, that host's frame renders blank with no error. If it is absent, any site may embed the page
Third-party cookie partitioning	Browsers restrict or block cookies for pages inside an iframe on a different domain. This is why session identity here cannot use cookies
Redis / key-value store	A very fast, simple database that stores values under names. Used here for session state, rate limits, and the spending counter
TTL (time to live)	An expiry set on stored data. Data with a TTL deletes itself, which is how session cleanup happens without any cleanup code
Prompt caching	Paying to store a long, unchanging chunk of prompt on Anthropic's side so later requests reuse it at one tenth the input price
Structured output	Requiring the model to answer in a defined shape (named fields) rather than free prose, so the application can read specific values reliably
Host page	Any page that frames the simulation and listens for its completion message. The embed harness is the one specified in the Test Plan; a Storyline module or an LMS course page would be another

---

3. System shape



The browser never talks to Anthropic. It talks only to the application's own API routes, which hold the key and enforce the limits.

The framed path is a retained capability, not the main delivery: today the simulation is reached by direct link from both static sites.

3.1 Request flow for one turn

1. Browser sends the learner's message and its session ID to `/api/turn`.
2. The route checks the spending ceiling and the rate limit. Either can divert to demo mode.
3. It loads the session (turn count, transcript, Marcus's state) from Redis.
4. It rejects the request if the session has already ended or is over 15 turns.

5. It assembles the performer request: cached system prompt, current state, conversation history, plus the final-turn instruction if this is input 15.
6. It calls the Anthropic API and receives structured output.
7. It records actual cost from the response's usage fields into the spend counters.
8. It saves the updated session to Redis and returns Marcus's dialogue and the call status to the browser.

When the status is terminal, the browser calls `/api/evaluate`, which reads the transcript from Redis, makes one evaluator call, and returns judgments and feedback. The application computes the score from those judgments (D-050) and stores the assembled report against the session (D-141).

---

4. Hosting: Vercel, with Next.js

Decision: a Next.js application deployed on Vercel.

The Scope Document already chose a standalone application host over a static page with a serverless proxy (D-003). This settles the two remaining questions: which host, and which framework.

Why Vercel over Netlify. Both would work, and this is not a close-run technical argument. Vercel is chosen because Next.js is its native framework, so the page and its API routes deploy together as one project with no glue; because environment variable management and preview deployments are straightforward in the dashboard; and because applications of exactly this shape (a chat UI plus a couple of API routes calling a model provider) are the most heavily documented pattern on the platform, which matters when the maintainer needs to search for an answer.

Why Next.js rather than a plain static page plus separate functions. The alternative is fewer concepts but more moving parts: a separate build for the page, a separate deployment for the functions, and cross-origin configuration between them. One framework, one deploy, one place to look is the better trade for a single-maintainer project.

Reversibility, deliberately preserved. Nothing in the design is Vercel-specific except the deployment configuration. The Redis choice in section 8 is host-agnostic on purpose, so moving to Netlify or elsewhere later means changing where it deploys, not rewriting it.

---

5. Model selection

Two roles with different requirements (D-008): the performer runs Marcus in conversation, the evaluator scores the finished transcript.

5.1 Current lineup and pricing

Model	API ID	Input	Output
Claude Fable 5	`claude-fable-5`	$10 / MTok	$50 / MTok
Claude Opus 5	`claude-opus-5`	$5 / MTok	$25 / MTok
Claude Sonnet 5	`claude-sonnet-5`	$2 / MTok	$10 / MTok
Claude Haiku 4.5	`claude-haiku-4-5-20251001`	$1 / MTok	$5 / MTok

5.2 The decisive asymmetry

The performer runs **up to fifteen calls per session**. The evaluator runs **one**.

That means capability bought for the evaluator is roughly fifteen times cheaper per unit of quality than capability bought for the performer. And the evaluator is where the project's highest-rated risk lives: scoring consistency (E1). Spending more on the single call that must be repeatable, and less on the fifteen calls that must be fast and characterful, is the shape the economics recommend.

5.3 Modelled cost per session (design-time estimate)

These figures are a model, made before anything was built, and they were wrong. The measured figures are in 5.3a. They are kept because the comparison between model pairings still holds, and because how the estimate went wrong is the useful part.

Modelled on a full fifteen-turn session with prompt caching enabled: a static system prompt (since measured at 5,448 tokens), learner messages averaging 55 tokens, Marcus's replies 95, structured output 120 per turn; evaluator input of roughly 5,100 tokens with 1,300 tokens of judgments and feedback.

Performer	Evaluator	Modelled cost per session	Sessions per $25
Sonnet 5	Sonnet 5	$0.085	294
Sonnet 5	**Opus 5**	**$0.120**	**208**
Opus 5	Opus 5	$0.212	117
Haiku 4.5	Sonnet 5	$0.054	462

Decision: Sonnet 5 as the performer, Opus 5 as the evaluator.

Sonnet 5 for the performer buys speed, which a conversation needs, at a quarter of Opus pricing across the fifteen calls where volume lives. Opus 5 for the evaluator costs an extra $0.035 per session, which is less than a third of a cent per hundred sessions of difference, and spends it precisely where consistency and careful reading matter.

Haiku 4.5 as the performer was considered and rejected for now. It is half the cost again, but the performer is carrying an unusually heavy instruction load: eight behavior triggers with precedence rules, a three-tier knowledge model that must never leak, a state ladder with movement exceptions, and a disclosure gate. Tier 2 leakage would destroy the simulation's entire purpose (D-029), and that is not the place to economise first. Worth revisiting after testing establishes whether the prompt holds.

Both models are set by environment variable, not hardcoded, so testing can compare them without a code change and a future deprecation is a settings edit rather than a rebuild.

5.3a Correction: the measured figures, and why the published ones were wrong five times

The figures in 5.3 were modelled, not measured, and they were wrong. This section records what a real session actually costs and why it took seven versions of this estimate to find out, because the reasons are more useful than the number.

The measured figures. Taken from a complete session run on production, on a phone, signed out, on 8 September 2026.

	Value
Full session: conversation plus evaluation	**$0.27**
Of which the evaluation, cold	~$0.09 to $0.11
Of which the performer, across the conversation	~$0.16 to $0.18
Modelled in 5.3	$0.12
Variance	**+125%**
Sessions inside a $25 month	**~92**
Learners, at two attempts each	**~46**

Later measurements, and a price correction (D-131, D-139). A seven-turn production session cost $0.15, so real sessions measured $0.14 to $0.27 depending on length. When the spend counter was checked against the Anthropic Console, the performer's half disagreed: the Sonnet 5 prices in the application's pricing table had been derived from a ratio rather than looked up, and were 1.5 times the published rates. The table in 5.1 shows the published rates. With them corrected the counter agreed with the Console, so the $0.16 to $0.18 performer figure above, taken from the counter before the correction, overstates the performer's share. The ceiling is still sized against the upper end of the measured range, because a ceiling that assumes the good case is not a control.

Cost here is a distribution, not a figure, and should always be quoted with a range. Across 41 evaluations the per-evaluation cost ranged from $0.0341 to $0.0956. The same transcript, scored twice through identical code, cost $0.0956 and then $0.0610. Adaptive thinking spends a variable budget per call, so a single measurement is a sample,

never a rate.

The cache is why five earlier estimates were wrong in the same direction. The consistency suite measured $0.0486 per evaluation across thirty runs and that number is real, but it describes a condition no learner will ever be in. Thirty evaluations running back to back share a warm prompt cache: the first pays a cache write, the other twenty-nine read it. The evidence is unambiguous; on the deployed path the first evaluation of a batch recorded 8,268 cache-write tokens and zero reads, and the other ten recorded zero writes and 8,268 reads each.

In production every evaluation is a cold write. A learner triggers exactly one evaluation per session, the ephemeral cache lives about five minutes, and nothing reads it back. Paying 125% of the base price for a cache nobody reads is worse than not caching at all. For the evaluator specifically, on a low-traffic artifact, prompt caching is a net loss.

The performer's caching profile is the opposite and should not be assumed to share the fault. A single session makes up to fifteen performer calls minutes apart, so its cache is written once and read many times, which is the case caching exists for. H2 and H3 verify that it is working.

No change is being made to the evaluator's `cache_control` on this evidence. Removing it would save roughly a cent per production session and roughly double the cost of every future test run.

The general lesson, which outlived the number. Six of the seven versions of this estimate were derived by reasoning from assumptions or measured under a condition that only exists during testing. A harness that runs thirty evaluations in four minutes is not a fast version of production; it is a different one. And a price that can be looked up should never be derived.

5.4 Realistic budget

Most sessions will not run the full fifteen turns; a strong learner may finish in seven and an off-topic ending stops at two. A blended average nearer 0.09*islikely*, *soa* * *25 monthly ceiling supports roughly 250 to 300 sessions**. (Superseded: see 5.3a. Real sessions measure $0.14 to $0.27, so the ceiling supports roughly 92 sessions at the upper end and more at the lower.)

Retries change the unit that matters. Since retries are permitted (D-070), sessions are not learners. If the average learner makes two attempts, 250 to 300 sessions is 125 to 150 people. (Superseded: see 5.3a. At the upper end of the measured range that is roughly 46 learners inside a $25 month.) That is still ample for a portfolio artifact and for a module cohort of any realistic size, but the figure to quote is learners rather than sessions, and the two differ by whatever the retry rate turns out to be. Retry rate is therefore worth tracking in telemetry (section 17), since it is the number that turns a session budget into a headcount.

---

6. Prompt caching

D-037 made caching a cost control rather than an optimization, and the numbers confirm it: caching the static system prompt cuts performer cost by **44%** as modelled and **45%** as measured (D-142).

The mechanics that matter here:

- The minimum cacheable prefix for Sonnet 5 is **1,024 tokens**. The performer's static system prompt is **5,448 tokens, measured**, so it clears the threshold by a factor of 5.3. This is worth watching: if the prompt is ever trimmed below that threshold, caching silently stops working and cost nearly doubles with no error.
- Cache reads cost **10%** of base input pricing; a 5-minute cache write costs 125%.
- The default 5-minute TTL is correct here. Turns within a session are seconds apart, and each read refreshes the lifetime at no cost. The 1-hour TTL costs twice the input price to write and buys nothing for this usage pattern.
- The cache breakpoint goes after the static block and before the per-turn dynamic block, which is exactly the split already specified in the design document (section 10.1). The static and dynamic halves of the prompt were separated for design reasons and turn out to be separated correctly for caching too.
- Response `usage` fields report `cache_read_input_tokens` and `cache_creation_input_tokens`. These are logged as telemetry so a caching regression is visible rather than silent.

---

7. Session state without cookies

Embedding in a frame on another origin means cookies are unavailable (Scope 6.3): browsers partition or block cookie access inside a third-party iframe, so anything built on cookies would work standalone and fail when framed, which is the worst kind of bug to find late. This was written with a Storyline module in mind; the same reasoning applies to any host page, and the property is retained (D-188).

Decision: session state lives in Redis, keyed by an opaque session ID that the browser holds without cookies or storage.

On start, `/api/session` generates a random session ID, writes an initial state record, and returns the ID. The browser keeps it in memory and in the page address's fragment (the part after `#`). A fragment is never sent to the server, needs no cookie or local storage, and survives a reload, which is what lets a learner who reloads, rotates the phone, or swipes back recover a finished report rather than lose it (D-126, D-141). Nothing here is something a browser privacy setting can take away inside an iframe.

The stored record holds: turn count, the transcript so far, Marcus's `intensity`, `tier1_disclosed`, `tier2_confirmed`, the off-topic count, the call status, and, once scored, the assembled report. A one-hour TTL means abandoned sessions delete themselves and no cleanup code is needed. **A stored report is returned as it is and never re-evaluated** (`regenerated: false` in the logs), because re-running the evaluator would spend money again and could return a different score for the same transcript.

An alternative was considered and rejected: a signed stateless token carrying the state in the browser, verified by signature on each request. It avoids a database dependency, but it requires cryptographic signing code, it is replayable (a client could resend an old token to reset its turn count), and closing that hole needs server-side storage anyway. Since Redis is already required for the spending ceiling, using it for session state is simpler, tamper-proof, and easier to reason about.

Server-held transcript is the security-relevant part. The conversation history lives on the server, not in the browser. If the browser sent the history back each turn, a modified client could forge Marcus's replies and manipulate its own evaluation. For a portfolio piece the stakes are low, but the design should not be embarrassing to explain.

---

8. Rate limiting and the spending ceiling

Storage: Redis reached over an HTTP REST API. Confirmed available on the project's provisioned database.

Chosen because it speaks HTTP rather than requiring a persistent connection, which suits serverless functions that start and stop constantly; because it offers atomic increment operations, which is what a spending counter needs to stay correct when two requests arrive at once; because its free tier is comfortably sufficient at this volume; and because it is not tied to Vercel, which keeps the hosting decision reversible (section 4).

The HTTP requirement is the part that matters, and it is not a preference. Serverless functions start and stop constantly, and a traditional Redis provider expects a long-lived TCP connection held open by a running server. Used from serverless, that pattern produces connection churn, exhausted connection pools under concurrency, and cold-start latency on every call. A Redis service that offers only a `redis://` connection string and no HTTP REST endpoint is the wrong shape for this architecture, and discovering that during the build is expensive.

Region matters more here than it usually would. The application makes several Redis calls per turn, so the database should sit in the same region as the Vercel functions. A database in one continent and functions in another adds latency to every single turn, which shows up as a conversation that feels sluggish for no visible reason.

8.1 Four layers of control

Layer	Mechanism	Purpose
Turn cap	Server rejects any turn past 15 for a session	Design constraint and cost constraint at once. Enforced server-side, so a modified client cannot bypass it
Rate limit	Six new sessions per IP address per hour	Stops one actor consuming the budget
Daily sub-	A daily counter, $2 by default, beneath the	Turns a drained budget into a one-day outage instead of a month-long

ceiling	monthly one	one (D-128)
Monthly ceiling	A monthly counter, incremented by actual cost after every model call	The hard stop

8.1a Tuning the session rate limit for retries

Retries are permitted and encouraged (D-070), which constrains the rate limit directly: it must not block a learner making a legitimate second or third attempt after reading their feedback. A limit of two or three sessions per hour would look reasonable in isolation and would break the intended learning loop.

Setting: six new sessions per IP per hour. Generous enough that a motivated learner retrying several times is never blocked, tight enough that a single actor cannot consume the monthly budget in an afternoon. Six sessions is at most $0.72 per hour per address against a $25 ceiling. (Superseded: at the measured $0.27 per session this is $1.62 per hour, and one actor could exhaust a $25 month in roughly fifteen hours rather than thirty-five. **The limit of six stays anyway.** D-070 wanted two things that are incompatible at $0.27: a spend cap of $0.72 an hour, and a generous enough allowance that a learner retrying is never blocked. The retry allowance wins, because the ceiling is what caps total spend and the rate limit only has to stop instantaneous abuse. See D-128, and the daily sub-ceiling in 8.1, which exists so that a drained month cannot leave the artifact in demo mode until the calendar turns over.)

A caveat worth recording: corporate networks put many people behind one address. A cohort working through the module simultaneously from one office could hit an IP limit that no individual deserves. This does not affect portfolio use, but it would matter for real deployment, and the honest note is that per-IP limiting is the wrong tool at that scale. The setting is an environment variable so it can be raised for a known cohort.

The window is measured from the first session in it. The counter's expiry is set only when it first reads 1, so attempts refused after the limit do not extend the hour (D-195). A rate-limited visitor is offered the recorded call with wording that makes clear the simulation is running normally for everyone else (D-198).

8.2 How the spending counters stay accurate

Every Anthropic response reports exactly how many tokens were used, including cache reads and writes. After each call the route computes the real dollar cost from those numbers and adds it to three counters: one for the day (`spend:day:2026-09-08`), one for the month (`spend:month:2026-09`), and one that never resets (`spend:lifetime`). This is measured spending, not an estimate, and it captures the caching discount automatically. The counters were verified writing on production and agreeing with the Anthropic Console once the price table was corrected (D-138, D-139).

Before a session starts, the route reads the counters. Over either ceiling, the request diverts to demo mode instead of reaching the API, and spends nothing (D-144). A session that began under the limit is allowed to finish, including its evaluation, because cutting a learner off at turn nine spends the money and delivers nothing.

8.2a Two ways the spending counter could fail silently

The counter is the control this project exists to preserve, so its failure modes deserve naming rather than assuming.

A missing counter must never be read as zero. At the start of a month the month key legitimately does not exist. Mid-month, a missing key means something went wrong: the data was evicted, the database was wiped, or the wrong instance is being read. Treating absent as zero would silently reset the budget and hand back exactly the unmetered spending the ceiling prevents.

Handling: the month and lifetime keys carry **no TTL** and are never allowed to expire; the day key expires on its own, which is how the daily ceiling resets. Redis eviction is **disabled** on the database, so keys are not dropped under memory pressure. `spend:lifetime` is incremented alongside and never resets; a month key that is missing while the lifetime key shows activity is a fault condition, and the application fails closed to demo mode rather than assuming a clean slate.

The application's ceiling is not the last line of defence. A hard spend limit set on the Anthropic account itself is an independent backstop that survives any failure of this application's own logic: a bug in the counter, a wiped database, a misconfigured environment variable. The application ceiling should sit below it, so the app's own graceful demo mode triggers first and the account limit is never reached in normal operation.

Setting both is the difference between a budget control and a budget intention. The devlin.ai failure happened because spending was governed by something outside the owner's control; the correction is not to trust a single mechanism inside it either.

8.3 Two failure decisions worth stating

If Redis is unavailable, the application fails closed to demo mode. It does not carry on unmetered. An unreachable counter means spending cannot be tracked, and an untracked budget is exactly the failure this project exists to prevent. The visitor still sees a complete artifact, which is what makes failing closed acceptable here. Verified by invalidating the Redis token on a preview (D-144).

IP addresses are used transiently and never stored. Rate limit keys are hashed and expire with their window. No IP is written to any persistent record.

---

9. Demo mode

Demo mode (D-024) serves a complete recorded playthrough with its score report when the live simulation cannot run. It triggers on:

- The daily sub-ceiling or the monthly spending ceiling being reached, each with its own message (D-143).
- Redis being unreachable (section 8.3).
- Not, at present, an Anthropic API error. Session creation makes no model call (Marcus's opening line is fixed), and a mid-conversation API error after one retry returns a clean, retryable error instead, with no turn consumed and nothing spent (D-144). Diverting that path to demo mode is a recorded Stage 5 item.

A visitor who has hit the hourly rate limit is offered the same recording, with wording that keeps the two situations distinct (D-198).

Behavior is identical in both launch contexts: explain the situation, encourage trying again later, and offer a button to view the complete recording. The demo is offered, never substituted silently. Verified on production on all three refusal paths (D-209) and, after a fix, inside a frame (D-195, D-196).

The recorded transcript is a static JSON file in the repository (`lib/demo-session.json`), not a database record. It loads with the page and works even if every backend dependency is down, which is the point: the fallback must have no dependency that can fail alongside the thing it is replacing. It is frozen fixture T2, a real captured session, with a report scored under the current rubric (D-143).

---

10. Framing headers (for any host page)

Originally titled "Framing headers for Storyline." Retitled because the requirement is the same for any host (D-188).

The single most likely silent failure in the embed path (Scope section 9). A blocked frame renders as an empty rectangle with no error message.

Two requirements:

1. **Set** `Content-Security-Policy: frame-ancestors` to permit the host page's origin, plus `'self'`. For the embed harness that is the harness's origin; for a Storyline module it is wherever the module is served.
2. **Do not set** `X-Frame-Options` **at all.** It is a legacy header that cannot express multiple allowed origins, and if a framework default or platform setting emits `DENY` or `SAMEORIGIN`, it will block the embed regardless of what the CSP header says. This is a common cause of an embed that works locally and fails in production.

As implemented (D-201). A `headers()` function in `next.config.ts` emits the policy on every path, built from `ALLOWED_FRAME_ANCESTORS`:

`ALLOWED_FRAME_ANCESTORS`	Header emitted
unset	`frame-ancestors *;`

`https://dbacon6093.github.io`	`frame-ancestors 'self' https://dbacon6093.github.io;`

`'self'` is added automatically. The variable accepts a space- or comma-separated list of origins, each with its scheme and no trailing slash. **It is read at build time**, so a change in the dashboard does nothing until the next deploy; the computed value is written to every build log. Only `frame-ancestors` is set: a fuller policy would need a `script-src` that Next.js's inline bootstrap scripts would break.

For six weeks this header did not exist (D-200, D-203). Framing worked anyway, because an absent `frame-ancestors` directive restricts nothing. Every embed test passed for that reason, which is why the Test Plan now says a frame that renders does not prove a policy permits it unless a restrictive policy is in place (Test Plan F4).

On permissiveness. The unset default is deliberately permissive rather than restrictive, so that a missing or mistyped variable can never produce a blank frame. That is acceptable for this application: the page has no authentication, no user data, and no privileged actions, so the classic clickjacking risks do not apply. The real risk of permissive framing is someone embedding the simulation and consuming the budget, and that is already controlled by the rate limit and the spending ceilings rather than by the header. **The deployed value is restrictive** (section 15.1), so a new host must be added to the variable, followed by a redeploy, or its frame will be blank.

---

11. Completion handoff to a host page

Originally titled "Completion handoff to Storyline." Retitled for the same reason as section 10; the payload and the call did not change.

On a terminal state the page calls `window.parent.postMessage()` with the computed results (D-028). The payload carries total score, pass or fail, completion status, outcome, per-criterion points, and the escalation count. The narrative feedback stays in the application (D-027).

The full payload specification and the host-side listener code are deliverable 5, the Embedding and LMS Readiness Note (file still named `storyline-integration-note`, D-186). Two implementation points belong here:

- The message specifies a target origin rather than `*`, so results are not broadcast to any page that happens to embed the simulation. **As implemented (D-219):** the target is `https://dbacon6093.github.io`, the same origin the framing allowlist permits (D-202), so the embed harness, the training site and the portfolio can receive the score and no other page can. A host on a different origin is added by setting `NEXT_PUBLIC_COMPLETION_TARGET_ORIGIN`, which is read at build time and needs a redeploy. **The failure is silent:** a browser drops a message whose target origin does not match the parent's, with no error, so a new host must be added to both this variable and `ALLOWED_FRAME_ANCESTORS`. A single value is supported; a second host would need a small code change. Until D-219 the code posted to `*` while this section said otherwise, the same shape of gap as the framing header in section 10.
- The listener on the host side must validate the sender's origin before acting. That code is in deliverable 5, with Storyline as the worked example and the embed harness (Test Plan section 9.1) as the verifying host.

---

12. Data retention and privacy

Data	Where it lives	How long
Conversation transcript	Redis, keyed by session ID	One hour TTL, then automatically deleted
Score and feedback	Returned to the browser, and stored in Redis against the session so a reload returns it (D-141)	Expires with the session record
IP address	Hashed into a rate limit key	Expires with the rate limit window; never stored in a record
Spending	Redis counters: day, month, lifetime	The day key expires; the month and lifetime keys are retained. They contain no conversation content
Aggregate telemetry	Redis counters	Retained; contains no conversation content

No transcripts or reports are logged or persisted beyond the session, and this is a deliberate choice rather than an omission. Storing them would be useful for debugging and would create an obligation to explain, protect, and eventually delete records of people's practice attempts. For a portfolio artifact the debugging value does not justify it. The report is kept for the life of the session only so that a learner does not lose feedback to a stray back gesture.

Telemetry is counts only: sessions started, endings by type, score distribution in bands, token usage, cache hit rates, and spending. Enough to know whether the thing is working and what it costs; not enough to reconstruct anyone's conversation.

The demo transcript is captured deliberately during testing, from a session run by the project owner, and stored as a static file with knowledge and consent. It is not harvested from a visitor.

---

13. Latency and streaming

Decision: no response streaming in V1.

Streaming (showing text as it is generated) reduces perceived wait, but it complicates structured output, since the application needs the state fields as reliable named values rather than parsed out of a text stream. With Sonnet 5 and roughly 120 output tokens, a Marcus reply should arrive in a small number of seconds, which is not unrealistic pacing for a phone conversation anyway. A typing indicator covers it.

The evaluator call is the longest wait in the application. Measured with adaptive thinking, it takes a median of roughly 18 to 23 seconds and up to about 31 seconds (D-109, D-115). This is why the evaluating state exists (D-026) and why it must show visible progress rather than a frozen screen. Whether the wait is uncomfortable is a question for the human pilot (Test Plan G7). If it is, the first two things to try are streaming the feedback narrative while judgments are computed, and revealing the score as soon as it is available with the narrative filling in after. Both are V2 candidates, not V1 scope.

---

14. Error handling

Failure	Response
Malformed structured output from the performer	Retry once. On a second failure, keep usable dialogue and carry the previous state forward rather than ending the session (Design 10.3)
Anthropic API error or timeout	Retry once with backoff, then a clean retryable error: Marcus produces no fabricated reply, the turn is not counted, and nothing is spent (D-144). Diverting this path to demo mode is a recorded Stage 5 item
Redis unreachable	Demo mode, failing closed (section 8.3)
Session not found or expired	Explain the session timed out and offer a restart
Turn submitted after a terminal state	Rejected server-side. The interface should already have disabled input (D-045); this is the backstop
Empty or whitespace-only input	Rejected in the browser, never submitted. It costs no turn and reaches no model (D-046). The server also rejects it as a backstop
Evaluator returns unusable output	Retry once, but only if enough of the function's time budget remains for the retry to finish (D-122, D-136). On a second failure, show the transcript with an honest message that scoring failed, rather than a fabricated score

The last row matters more than its size suggests. A scoring system that invents a number when it fails is worse than one that admits failure, and on a portfolio piece the failure mode a reviewer might see should be an honest one.

14.1 Errors are logged, never silently swallowed

Every `catch` block logs the underlying error server-side before returning its user-facing response. A handler that catches an exception and returns a generic message without recording the cause makes several genuinely different failures indistinguishable.

This is not hypothetical. During stage 1, `/api/session` caught every failure and returned "session unavailable" with nothing written to the logs. A missing environment variable, an incorrect credential, an unreachable host, and a permissions refusal all produced identical symptoms, and the runtime logs were empty because the error never reached them. Diagnosis required editing the code to reveal what it already knew.

The user-facing message stays vague, which is correct: a visitor does not need the internals. The log entry carries the detail. The two are separate concerns and should not be collapsed.

Diagnostic detail may be returned in the response body during development stages, as a temporary convenience, but is removed at stage 5 when demo mode takes over these handlers. Server-side logging is permanent.

---

15. Configuration

Everything environment-specific is an environment variable in the Vercel dashboard. Nothing sensitive is in the repository. **Vercel injects these into a deployment, so changing any of them requires a redeploy before the running functions see the new value** (D-207).

Variable	Purpose
`ANTHROPIC_API_KEY`	The key. Never sent to the browser. Set an independent spend limit on the Anthropic account itself as a backstop (section 8.2a)
`PERFORMER_MODEL`	Default `claude-sonnet-5`
`EVALUATOR_MODEL`	Default `claude-opus-5`
`PERFORMER_THINKING`	Thinking is disabled for the performer by default, because thinking tokens count against the output limit and emptied the most demanding turns (D-103). A low-effort adaptive setting is available if character quality regresses
`EVALUATOR_THINKING`	Adaptive. Disabling it made short transcripts score identically and long ones unreliably, so it stays adaptive (D-134 to D-137; Evaluation Design 8.2)
`MONTHLY_BUDGET_USD`	The monthly spending ceiling. 25
`DAILY_BUDGET_USD`	The daily sub-ceiling. Default 2 (D-128)
`SESSIONS_PER_IP_PER_HOUR`	The session rate limit. Default 6. Configurable so it can be raised for a known cohort or a tester (D-206)
Model prices	Every per-token price used by the spend counter is a configuration value, so a price change or a correction is a dashboard edit rather than a code change (D-139)
`UPSTASH_REDIS_REST_URL` / `_TOKEN`	Redis connection. The token must be the read-write one; the read-only token permits reads and refuses every write (D-090)
`ALLOWED_FRAME_ANCESTORS`	Hosts permitted to embed the simulation. Read at build time (section 10)
`NEXT_PUBLIC_COMPLETION_TARGET_ORIGIN`	The one origin the completion message is posted to. Default `https://dbacon6093.github.io`. Read at build time; set it, and add the host to `ALLOWED_FRAME_ANCESTORS`, to deliver the score to a host on another origin (section 11, D-219)
`MAX_TURNS`	Default 15. A setting rather than a constant, so testing can vary it
`STAGE_DEBUG`	Returns Marcus's state on every turn and opens the test-fixture seam, together with a separate shared secret (D-121). **Never set on Production**, and a preview that has it set keeps Deployment Protection on, because a learner must never see Marcus's intensity (D-025, D-080, D-212)

15.1 Confirmed deployment settings

Setting	Value
Host	Vercel, on the project's own subdomain. No custom domain
Repository	GitHub, **private**
`MONTHLY_BUDGET_USD`	25
`DAILY_BUDGET_USD`	2
`PERFORMER_MODEL`	`claude-sonnet-5`
`EVALUATOR_MODEL`	`claude-opus-5`

Production URL	`https://diagnose-before-solving.vercel.app`
`ALLOWED_FRAME_ANCESTORS`	`https://dbacon6093.github.io`, which emits `frame-ancestors 'self' https://dbacon6093.github.io;` (D-202). This covers the embed harness, the training site and the portfolio, which share that origin

15.2 The production domain must be stable, and it is visitor-facing

Two properties of the deployment URL matter beyond convenience.

It must be the production domain rather than a per-deployment address. Vercel issues each individual deployment its own URL containing a random segment, of the form `project-a1b2c3d4-team.vercel.app`. These are valid and useful for previewing a specific build, and they change on every push. Anything that hard-codes one, including a host page's iframe address, the training site's "Start the call" link, and the `APP_ORIGIN` in a host's listener, works when tested and fails silently at the next deploy.

It is read by the audience. With no custom domain, the Vercel project name becomes the address a hiring manager sees in their browser. A project named for the work reads as considered; a project named for the repository scaffolding does not. Renaming the Vercel project is trivial before anything references the URL and progressively less so afterward, since the training site, the portfolio page, and the frame-ancestors setting all point at it.

Renaming a project does not retroactively rename existing deployments. Per Vercel's documentation, new deployments generate URLs using the updated name; deployments made before the rename keep their old addresses, and Vercel does not guarantee how those pre-rename URLs continue to behave. A redeploy is therefore required for the new name to appear, and the authoritative value is whichever domain the project's Settings → Domains page marks as production after that redeploy.

A constraint this creates, and the reason to record it. Because the project uses a `.vercel.app` subdomain rather than a custom domain (D-083), the production URL is a function of the project name. Vercel's own guidance recommends a custom domain for production-facing applications precisely because generated URLs change with renames. The training site's "Start the call" link already has this address in it (D-177, D-190), and any host page that frames it would too, so **the project name is effectively frozen** now that those are published: a later rename silently breaks the link and any completion handoff.

A custom domain remains the durable answer and can be added later without breaking anything, since a custom domain can be pointed at the same project regardless of its name. That is the migration path if this constraint ever becomes inconvenient.

15.3 A consequence of the private repository

The code is not a portfolio artifact. A reviewer cannot read it, which means the documents carry the full weight of demonstrating how the work was done.

This is not an argument for making the repository public; a private repository is a reasonable default and removes a class of accidents involving secrets. It does mean the case study should carry a small number of well-chosen code excerpts, most usefully the scoring module and the performer prompt, so that a reader can see the craft rather than only its description.

The prompts live in their own plain text files in the repository, separate from application logic. This is a deliberate maintainability choice: the owner will want to adjust Marcus's wording after testing, and that should never require touching code that could break the application.

---

16. Repository shape

```
/app
  page.tsx                  chat UI, results view, demo mode
  /api/session/route.ts     create a session
  /api/turn/route.ts        one exchange with Marcus
  /api/evaluate/route.ts    score the finished transcript, store and return the report
/lib
  anthropic.ts              API calls, caching, usage accounting
  redis.ts                  session state, stored report, rate limits, spend counters
  evaluator.ts              the evaluator call, output validation, citation repair
  scoring.ts                computes totals from evaluator judgments
  pricing.ts                per-token prices for the spend counter
  demo-session.json         the recorded playthrough and its report
  prompts/
    performer.md            Marcus's system prompt
    evaluator.md            the evaluator prompt
    performer-duration-note.md   Marcus's wording for the learner's stated duration (input 11)
    performer-final-turn.md      the instruction added on the last input
/scripts                    test harnesses, unit tests, frozen reference transcripts
next.config.ts              security headers
```

Scoring arithmetic lives in `scoring.ts` as ordinary code, per D-050, and has unit tests. It is the one piece of logic in the application where correctness is checkable by example, and the score is what a learner will dispute.

---

17. Monitoring

Worth watching after launch, all available from the telemetry counters:

- **Monthly spend against ceiling.** The number that matters.
- **Cache hit rate.** A drop means the prompt changed and caching stopped working; cost roughly doubles silently.
- **Ending distribution.** A sudden rise in off-topic endings suggests the two-strike rule is firing on legitimate messages.
- **Score distribution.** Everyone passing or everyone failing suggests a rubric or performer problem, not a population of unusually good or bad learners.
- **Learner retry rate.** How many sessions are second or later attempts. This is what converts a session budget into a number of people, and it is also a learning signal: a high retry rate after a low first score is the loop working.
- **Error and API retry rates.** Rising retries against the model provider mean something upstream changed.

---

18. What the owner maintains, and what to get help with

An honest division, given the stated experience level.

Safe to change without help:

- Marcus's wording and the evaluator's wording, in the prompt files. Text only.
- The budget ceilings, the rate limit, models, prices, and allowed frame hosts, in the Vercel dashboard, followed by a redeploy.
- The demo transcript JSON, replacing it with a better recorded playthrough.
- Copy in the landing and results views.

Get help before touching:

- Anything in `redis.ts`. The spending counter is what stands between this project and the failure that killed the last one.
- The security headers in `next.config.ts`. A wrong edit here breaks any host page's embed silently.
- The API route logic, particularly turn enforcement and the terminal-state checks.
- The `postMessage` payload shape, once any host page is built against it.

Rotate the API key immediately if it ever appears in a commit, a screenshot, a shared log, or a support conversation. Rotation is a dashboard action and a redeploy; there is no reason to hesitate over it.

19. Risks

Risk	Mitigation
Embed fails silently when framed by a host page	Verify with the embed harness against the production address, not a preview deployment (Test Plan F3 to F11; Scope section 9). Verify `X-Frame-Options` is absent, not merely permissive. Storyline's player and LMS wrappers are not covered by the harness and would need their own test if one is ever built (Test Plan section 17)
Prompt edited below the 1,024-token caching threshold	Cache hit rate is monitored; a drop to zero is the signal
Model deprecation	Model IDs are environment variables, so a replacement is a settings edit; until then an API failure surfaces as a clean, retryable error rather than a broken page
Redis free tier limits exceeded	Volume here is far below them; spend telemetry would show it first
Spend counter evicted or wiped, silently resetting the budget	No TTL on month and lifetime keys, eviction disabled, a never-resetting lifetime counter as a cross-check, and an independent spend limit on the Anthropic account (section 8.2a)
~~Redis provider offers no HTTP REST endpoint~~	**Closed.** REST endpoint confirmed available before the build
A later project rename silently breaks the training site's link and any published host page	The name is frozen now that those are published (section 15.2). A custom domain is the migration path if it must change
Cost per session higher than modelled	It was, by 125% at first (section 5.3a). The ceilings are hard stops regardless of whether the model was accurate. That is the point of a ceiling rather than a forecast
Session state grows large in Redis	Transcripts are capped at 15 turns and expire in an hour
A single visitor exhausts the budget	Rate limiting, the daily sub-ceiling, and the two-strike off-topic ending (D-046), which cuts unserious sessions to two turns
A debug preview is reachable by the public	`STAGE_DEBUG` is never set on Production, and a preview with it set keeps Deployment Protection on (section 15)

20. Open questions

1. ~~**The origin of the first host page, for `frame-ancestors`.**~~ **Resolved.** The embed harness, on `https://dbacon6093.github.io` (D-202).
2. **Whether Haiku 4.5 can carry the performer prompt.** Half the performer cost again. Worth a comparison run against Suite A, but not worth risking Tier 2 leakage to save money (Test Plan calibration question 5).
3. ~~**Sampling temperature for both roles.**~~ **Closed by the platform.** Temperature is deprecated on these models (D-101). The nearest available control, thinking configuration, is set per role in section 15.
4. **Whether the evaluating wait is uncomfortable enough to justify streaming in V1.** Answered by the human pilot, not by argument (Test Plan G7).

Appendix: cost model assumptions

Reproduced so the figures in section 5.3 can be checked or recalculated when pricing changes.

Assumption	Value
Performer static system prompt	**5,448 tokens**, cached (measured 23 September 2026; the original estimate of 2,000 was nearly three times too low, see below)
Learner message, average	55 tokens
Marcus reply, average	95 tokens

Performer structured output per turn	120 tokens
Turns per full session	15
Evaluator input (prompt plus transcript)	~5,100 tokens
Evaluator output (judgments plus feedback)	1,300 tokens
Cache write multiplier, 5-minute TTL	1.25x input
Cache read multiplier	0.10x input

Sonnet 5 performer with caching: $0.062 per full session, against $0.111 uncached. Opus 5 evaluator: 0.058*persession*.*Combined* : **0.120**. (Superseded: see 5.3a. Real sessions measure $0.14 to $0.27.)

Measured, 23 September 2026 (D-142). The performer's static system prompt is **5,448 tokens**, not the 2,000 assumed above. It was read directly from `cache_creation_input_tokens` on the first turn of a production session, which is the exact quantity being cached and therefore not an estimate at all.

The other assumptions in this table remain estimates and are marked as such by omission. None of the remaining rows has been measured against a real session. They should not be treated as findings.

This does not change what the artifact costs, because nothing downstream depends on the table. The spend counter prices real token counts reported by the API, and it was validated against the Anthropic Console in H4 (D-139). The table exists so the section 5.3 figures can be recalculated when pricing changes, and an input three times off would have made it useless for that purpose while looking authoritative.

Two claims in section 6 are confirmed by the same measurement. The 1,024-token caching threshold for Sonnet 5 is cleared by a factor of 5.3, so the silent-caching-failure risk noted there is remote and the prompt has been growing rather than shrinking. And the stated 44% cost saving from caching the static prompt measures at **45%** on that session: $0.0204 with caching against $0.0373 without, priced with the corrected table. That figure was asserted during design and is now confirmed on real traffic.